

★ Member-only story

CLAUDE.md Best Practices: The Complete Guide for 2026



IAKH Studio

Follow

4 min read · Apr 22, 2026



14



1



1



If you use Claude Code, `CLAUDE.md` is the single most powerful lever you have. It's the file Claude reads at the start of *every* conversation — your chance to onboard an AI agent into your codebase before it writes a single line of code.



What Exactly Is CLAUDE.md?

CLAUDE.md is a special Markdown file that Claude Code automatically loads at the beginning of every session. Think of it as a persistent system prompt specific to your project — it tells Claude your tech stack, coding conventions, architectural patterns, and the rules it must follow throughout the entire engagement. Unlike regular documentation, it's not just for humans; it actively shapes how the AI reasons about and modifies your code.

The Three-Layer Structure

The most battle-tested structure for CLAUDE.md organizes content into three layers:

- **The What** — Your tech stack, project structure, and key dependencies (e.g., “This is a Next.js 15 app using Supabase and Tailwind CSS”)
- **The Why** — The purpose of key components and architectural decisions (e.g., “We use Zustand for state because Redux was too verbose for our scale”)
- **The How** — Explicit rules for how you want Claude to work (e.g., “Always write tests before implementation,” “Use ES modules, not CommonJS”)

The Golden Rule: Less Is More

The most common mistake engineers make is writing a `CLAUDE.md` that's too long. Anthropic's own official guidance says: if your file is too long, Claude ignores half of it — important rules get buried in noise. Boris Cherny, Anthropic engineer and the creator of Claude Code, keeps his own personal `CLAUDE.md` **under 100 lines**.

The filter to apply to every line is ruthlessly simple:

“*Would removing this cause Claude to make mistakes?*”

If the answer is no, delete it. General community consensus recommends staying under 300 lines, with shorter being better.

What to Include

Here's what actually belongs in your `CLAUDE.md`:

- **Bash commands** — Build, test, lint, and deployment commands Claude can't infer on its own
- **Code style guidelines** — Formatting rules, naming conventions, framework preferences
- **Safe-change rules** — Files or modules that should never be modified without explicit approval
- **File placement conventions** — Where components, utilities, and tests should live
- **Key architectural patterns** — State management approach, API design decisions, folder structure
- **Common gotchas** — Non-obvious behaviors that have burned your team before

What to Leave Out

Equally important is knowing what *not* to put in the file:

- Self-evident practices like “write clean code” or “add comments”
- Domain-specific knowledge only relevant to *one occasional workflow* — use Skills for that instead
- Instructions Claude already follows correctly without being told
- Overly specific schema or database instructions unrelated to most tasks

Use `/init` as Your Starting Point

You don't have to write `CLAUDE.md` from scratch. The `/init` command analyzes your codebase — detecting build systems, test frameworks, and code patterns — and generates a solid first draft. From there, you refine it collaboratively: chat with Claude, identify gaps, and commit the final version to version control so your entire team inherits the same AI behavior.

Sub-CLAUDE.md Files for Monorepos

For larger projects or monorepos, a single root-level `CLAUDE.md` is rarely enough. The best practice is to create **nested** `CLAUDE.md` **files** in subdirectories — so a `/backend` folder can have its own file with API-specific rules, while `/frontend` has component conventions. This keeps each file focused and short, while still giving Claude precise context no matter where it's working in the codebase.

Treat It as a Living Document

Your `CLAUDE.md` should evolve alongside your project. When Claude repeatedly makes the same mistake despite having a rule, it's a signal — either the rule is buried too deep and the file needs pruning, or the phrasing is ambiguous and needs rewriting. Anthropic engineers internally run `CLAUDE.md` content through prompt-improvement tools and commit updates team-wide to maintain consistency.

CLAUDE.md vs. Skills

When Loaded

- CLAUDE.md: Every session, automatically.
- Skills (SKILL.md): On demand or when relevant.

Best For

- CLAUDE.md: Universal project rules.
- Skills (SKILL.md): Occasional, specialized workflows.

Risk of Overuse

- CLAUDE.md: Bloats context and buries key rules.
- Skills (SKILL.md): Minimal, as they are loaded selectively.

Example Content

- CLAUDE.md: Code style and build commands.
- Skills (SKILL.md): Database migration guides and deployment runbooks.

The Debug Signal You Shouldn't Ignore

Here's a reliable diagnostic: if Claude keeps asking you questions that are already answered in your `CLAUDE.md`, the phrasing is ambiguous. If Claude

keeps repeating a mistake despite a rule against it, your file is likely too long and the rule is getting lost. These are not Claude failures — they are CLAUDE.md failures, and they're always fixable. A great CLAUDE.md is one where Claude *never* has to ask what it already should know

Claude Code

Claude Md

AI

AI Agent

Antrophic



Written by IAKH Studio

275 followers · 322 following

Studio

Follow

